

People's Priorities

Civic Grievance & Constituency-Intelligence Platform

A citizen-to-government platform that turns raw local complaints into a prioritised, explainable action queue for officials — and rolls those complaints up into a data-driven view of what each ward actually needs.

Hackathon	Build with AI: Code for Communities (Google DevRel)
Track	People's Priorities — grievance redressal & constituency planning
Timeline	3-day build to a live, working prototype (slides alone will not pass)
Constraint	Built from scratch · Google tech mandatory · original / not previously deployed
Stack	Flask + SQLite (MySQL-portable) · Gemini / Vertex AI · Google Maps Platform
ML posture	Rule-based core first; embeddings/ML clustering added last, only if time allows

1. Executive Summary

We are building a two-sided civic platform. On the **citizen side**, anyone can report a local problem — a pothole, a dead streetlight, an overflowing bin, no water — in plain language (or their own language, or by voice), get a ticket, and track it. On the **official / planner side**, those reports arrive already understood, categorised, and ranked by a transparent urgency score, plotted on a live map, and — the headline — aggregated into a per-ward picture of the top issues over time.

The track is named for *constituency planning*, and that is exactly the part most teams will skip: they will ship a complaint tracker and stop. Our winning angle is to go one step further — group duplicate reports so fifty complaints about one dark street become a single high-priority item reading “1 issue, 50 citizens affected,” and roll grievance themes up into a ward-level agenda. That is genuine intelligence, not a wrapper around an API call.

It is feasible in three days because Gemini does the language heavy-lifting, the data can be realistically simulated (no waiting on a flaky government feed), and the core urgency and duplicate logic works rule-based first — fully demoable on its own — with the machine-learning upgrade reserved for the end as an enhancement rather than a dependency.

2. The Problem & The Opportunity

Local civic problems are reported into a black hole. The pain sits with three groups:

- **Citizens** have no reliable channel to report issues and no way to know if anyone is acting. Complaints are lost, duplicated, or ignored.
- **Officials** are handed a pile of unstructured text with no way to triage. What is urgent? What is a genuine mass problem versus one loud voice? They cannot tell at a glance.
- **Elected representatives and planners** have no data-driven picture of what their area needs. Budget and attention get allocated on instinct, not evidence.

The opportunity is a single system that closes this loop: structured intake, automatic triage, a clear priority queue, and — crucially — the aggregation layer that converts thousands of individual complaints into **“the top three issues in Ward 4 this month, and they are rising.”** That is the difference between a complaint box and civic intelligence.

3. What We're Building (Product Overview)

The two sides

- **Citizen app:** submit a grievance (text / regional language / voice), receive a ticket ID, and check status through a simple chatbot.
- **Official & planner console:** a live, urgency-ranked queue with filters, a map of active issues, duplicate-clustering, and a ward-level planning view.

The core loop, end to end

1	Citizen submits a grievance in natural language.
2	Gemini understands it — category, a clean summary, and structured signals (safety risk, how many affected, recency).
3	The system scores urgency transparently and checks for duplicates of existing reports.
4	It lands live on the officials' dashboard and map, ranked by priority.
5	The citizen tracks status via chatbot; officials update it.
6	The planning view aggregates everything into per-ward themes and trends — the constituency-planning payoff.

4. Why This Track (Decision Rationale)

With three days, from scratch, and a live prototype as the pass/fail bar, the deciding factor is **fewest from-scratch hours to something that actually runs** — not which track has the flashiest ML story. On that test, People's Priorities wins:

- **Gemini does the hard part.** A retrieve-and-generate pipeline is largely API orchestration, not model training — the fastest of the four tracks to a working demo.
- **No dependency on flaky government APIs.** Grievance data can be realistically simulated and is fully under our control, which is demo-safe. CleanAir's CPCB feed is the opposite.
- **Google-native by default.** Gemini is the core engine here, not a bolt-on, which scores directly on the AI-implementation criterion.

The honest tradeoff: a RAG-style build can look thin if the “intelligence” is just retrieval plus paraphrasing. We neutralise that by adding real signal — explainable urgency scoring and duplicate clustering — so it reads as engineering, not a wrapper. That differentiation is the whole plan.

5. Architecture & Tech Stack

One Flask application serves both the citizen intake and the officials' console. Data lives in SQLite for a zero-setup start; the schema is written so it moves to MySQL unchanged if we want. The “live” feel on the dashboard comes from lightweight polling — no heavyweight realtime infrastructure to fight in a three-day window.

Layer	Technology	Why	Google ?
Intake + console	Flask + HTML/JS templates	One codebase, two views; fast to stand up; no framework overhead.	—
Datastore	SQLite (MySQL-portable)	No server to install; ships with Python; schema ports to MySQL later.	—
AI core	Gemini via Vertex AI	Classification, structured extraction, urgency signals, citizen chatbot, summaries, translation.	Yes
Map	Google Maps Platform	Grievance markers + ward-level heatmap; near-free to add.	Yes
Realtime feel	Polling (JS fetch)	Dashboard refreshes every few seconds — “live” without extra infra.	—
Differentiator	Embeddings + clustering	Duplicate/theme detection. Rule-based first; ML upgrade last, if time.	Opt.
Analytics (opt.)	BigQuery	Only if planning aggregates get heavy; otherwise compute in-app.	Yes
Realtime (opt.)	Firebase Firestore	Swap-in for true push updates if the polling demo feels weak.	Yes

Locked decision: Flask + SQLite as the core (you know it cold, it runs in minutes). Firebase and BigQuery stay as optional swap-ins so Google-tech purity never costs us working software.

6. Feature Breakdown

Everything is bucketed so scope can be cut cleanly under time pressure without breaking the demo.

Tier	Feature	What it does
CORE	Intake form	Citizen submits a grievance; gets a ticket ID.
	Gemini classification	Auto-detects category, cleans a summary, extracts structured signals.
	Explainable urgency	Transparent 0–100 score with visible reasons — not a black-box guess.
	Officials' dashboard	Urgency-ranked queue with ward / category / status filters.
	Status tracking + chatbot	Citizen checks status; officials advance it; full timeline kept.
	Live map	Active grievances plotted on Google Maps.
	Planning view	Per-ward top issues and trends — the track's namesake.
EDGE	Duplicate clustering	Many reports of one issue collapse to a single item: “1 issue, N affected.”
	Urgency explainability	Shows <i>why</i> something ranks high — survives a judge's “why?”
	Ward planning intelligence	Themes roll up per ward with trend-over-time; a real agenda, not a list.
	Multilingual + voice intake	Accept Bengali / Hindi and voice input — nails the community/inclusion spirit.
LATER	ML clustering upgrade	Embeddings + similarity replace the rule-based dedup (added last, if time).
	SLA-breach flagging	Predict which grievances will blow past their resolution deadline.

CORE must ship for a complete demo. **EDGE** is the signature layer that lifts this above a template. **LATER** is the ML enhancement — valuable, but never on the critical path.

7. Data Model

Two tables keep it lean. The clustering columns are filled in later by the duplicate-detection step; until then every report is its own item.

Table	Key columns
grievances	ticket_id · citizen_name · citizen_phone · raw_text · language · category · urgency_score · urgency_level · status · ward · latitude · longitude · cluster_id · affected_count · created_at · updated_at
status_history	grievance_id · status · note · changed_at — the timeline that powers the tracking view and the citizen chatbot

Indexes on ward / category / status keep the dashboard queries fast. Written in plain SQL so the whole schema lifts into MySQL without changes.

8. Synthetic Data Strategy

The demo has to look alive from the first second, and we can't wait on a government feed. So we generate a realistic dataset we fully control:

- **~330 grievances** spread across eight real Kolkata wards (Salt Lake, New Town, Behala, Shyambazar, Jadavpur, Ballygunge, Tollygunge, Garia) with real landmarks, over the last ~90 days.
- **Planted duplicate storms** — e.g. ~35 reports of one dark street in Salt Lake, ~25 “no water” in Behala, ~20 garbage in Shyambazar — so the clustering feature has something real to find.
- **Rule-based urgency baked in:** category severity + risk keywords (accident, live wire, child, dengue) so scores are sensible even before Gemini touches them.
- **Multilingual sprinkles** (a few Bengali / Hindi reports) to demonstrate language handling.
- **Realistic status spread** with timelines, so the tracking view and “resolved vs open” stats look genuine.

9. The 3-Day Build Plan

Each day ends on a hard “**Done when**” gate. We do not advance until the gate is met — that is how we avoid the day-3 debugging spiral.

Day 1 — Foundation, intake & classification

Block	Task	Done when
Morning	Project skeleton (Flask + SQLite). Schema for grievances + status_history. Synthetic data generator with planted duplicate clusters.	One command seeds ~330 realistic grievances into the DB.
Midday	JSON API: list / detail / stats endpoints with filters.	Hitting /api/grievances returns ranked, filterable data.
Afternoon	Citizen intake form (submit → ticket ID). Wire Gemini for category + summary + structured signals; compute rule-based urgency with visible reasons.	A submitted grievance appears classified, with an urgency score and its reasons.

Day 2 — Dashboard, map, clustering & chatbot

Block	Task	Done when
Morning	Officials' dashboard: urgency-ranked queue, filters, stat cards. Polling for live refresh.	Dashboard shows the live, ranked queue and updates on new submissions.
Midday	Google Maps layer: grievance markers + ward heatmap.	Active grievances plot correctly on the map by ward.
Afternoon	Rule-based duplicate clustering (collapse near-identical reports; set affected_count). Citizen status chatbot over the grievance timeline.	A duplicate storm shows as one item “N citizens affected”; a citizen can check status.

Day 3 — Planning view, polish, rehearsal (+ ML if time)

Block	Task	Done when
Morning	Planning view: per-ward top-3 issues + trend over time. Multilingual / voice intake if on track.	Selecting a ward shows its ranked themes and a rising-issues trend.
Midday	End-to-end polish; seed tuning so the demo narrative lands; fix rough edges.	Full loop runs start-to-finish without touching code.
Afternoon	Rehearse the 5-minute demo; buffer for the inevitable bug. Only if time remains: swap rule-based dedup for embeddings clustering.	You can present the whole flow smoothly; ML upgrade merged only if it's stable.

10. Google Technology Mapping

Google tech is mandatory. Here is exactly where each product earns its place — with Gemini carrying the AI-implementation weight rather than being decorative.

Product	Where it's used	Why it scores
Gemini / Vertex AI	Category detection, clean summaries, structured signal extraction, citizen chatbot, translation.	The AI engine, not a bolt-on — directly answers the 25% AI-implementation criterion.
Google Maps Platform	Grievance markers + ward-level heatmap on the console.	Turns data into a spatial story judges can see; near-free to add.
BigQuery (optional)	Backing store for the planning aggregates if they grow heavy.	Signals a real, scalable data layer for the deployment/scalability criterion.
Firebase (optional)	True push-updates if we want the dashboard to feel more alive than polling.	Extra Google-native surface + a stronger live-demo moment.

11. Judging Rubric Alignment

How the build maps to what judges score. AI implementation is the heaviest single lever at 25%.

Criterion	Weight	How we win it
AI implementation	25%	Gemini for language understanding + real signal on top (explainable urgency, duplicate clustering) — engineering, not a wrapper.

Criterion	Weight	How we win it
Problem fitment	High	A concrete, undeniably local Indian civic problem; multilingual + voice makes it genuinely inclusive.
Deployment / scalability	High	Clustering cuts official workload as volume grows — a real scaling story; MySQL/BigQuery-ready data layer.
Innovation / differentiation	Medium	The planning-intelligence layer is what most teams skip; it's the track's namesake and our headline.
Demo / execution	Pass/fail	A fully working live loop beats a half-working ambitious one every time; “Done when” gates protect this.

12. Demo Script (5-minute flow)

Lead with the intelligence, not the form. The order that lands:

1	Open on the planning view — “Here’s what Ward X actually needs this month, and it’s rising.” Establish the payoff first.
2	Show the duplicate cluster — one item, “50 citizens affected.” Explain the workload it saves officials.
3	Open a grievance — show the transparent urgency score <i>and its reasons</i> .
4	Submit a live grievance (ideally in Bengali / by voice) — watch Gemini classify it and it appear on the dashboard and map.
5	Check its status via the citizen chatbot to close the loop.

13. Risks & Mitigations

Risk	Likelihood	Mitigation
Gemini API setup / quota friction	Medium	Get the key on Day 0 (see prerequisites). Cache responses; rule-based urgency works even if Gemini is down.
Maps billing / key delay	Medium	Enable billing on Day 0. Fallback: a static ward heatmap image if the live map stalls.
Scope creep on “planning”	High	Hard-scope it to top-3-issues-per-ward + trend. No policy simulation or analytics beyond that.
ML upgrade eats Day 3	High	ML is strictly last and optional. Rule-based dedup ships regardless; ML merges only if stable.
Losing work again	Low	Git repo + commit at every “Done when” gate; push to GitHub after each day.

14. Prerequisites (do before Day 1)

Front-load the account setup so Day 1 is pure building:

- **Google Cloud project** with billing enabled (needed for Gemini via Vertex AI and Maps).
- **Gemini API key** (AI Studio or Vertex AI) tested with one hello-world call.
- **Google Maps Platform API key** with the Maps JavaScript API enabled.
- **Python 3.11+ environment** and a fresh **Git repository** (push to GitHub after each day).

- **Optional:** a Firebase project, only if we decide to swap polling for push updates.

15. Positioning — The Pitch That Wins

“Most civic apps are a complaint box — you shout into it and hope. **People's Priorities** is different. It *understands* every complaint in the citizen's own language, tells officials not just what's wrong but *why it's urgent*, recognises when fifty people are reporting the same broken street and treats it as one high-priority issue, and rolls it all up into a live agenda of what each ward actually needs. It turns noise into a plan.”

Locked for this build: People's Priorities track · Flask + SQLite core · Gemini + Maps as the mandatory Google tech · rule-based urgency & dedup first · ML clustering last and optional. Next action after approval: Day 1 morning — project skeleton + synthetic data generator.